Developer DNA Matrix (DDM):

Behavioral Intelligence Framework — Phase 2: Deep Learning

Advanced Machine Learning & Deep Learning — Dataset: 6,000 Developers — Phase 1 Best: SVM RBF (F1 = 0.8116) — Phase 2 Best: MLP (F1 = 0.8164)

Abstract.

Phase 1 established an ML-only baseline of F1 = 0.8116 using a Support Vector Machine with RBF kernel, and identified that the fixed 30% Proof-of-Work weight in the Developer DNA Score formula creates systematic misclassification when volume and quality signals disagree. Phase 2 addresses this limitation by replacing the fixed kernel with a **Multi-Layer Perceptron (MLP)** capable of learning non-linear combinations of the same 28 engineered features. The MLP (128 → 64 → 3, BatchNorm, Dropout, early stopping) achieves **macro-F1 = 0.8164** and **Accuracy = 0.815** on the held-out test set, surpassing the SVM baseline by +0.0048 F1 points. An ablation study confirms that both BatchNorm and Dropout contribute positively. All figures, notebooks, and data files are included in the project repository.

1 Introduction & Phase 2 Motivation

Phase 1 of the Developer DNA Matrix project (DDM) demonstrated that GitHub behavioral signals — formalized as 23 raw signals and 28 engineered features across five DNA dimensions — can classify developer competency tiers with macro-F1 = 0.8116 (SVM RBF). Failure analysis in Phase 1 identified the root cause of the remaining 19% error rate: the *fixed* DDS weighting formula (Eq. 1) assigns 30% of the score to Proof-of-Work volume. When a developer's volume signals and quality signals conflict — an expert who has low commit count but high test coverage, or a beginner with high commit count but no tests — the kernel distance metric is pulled toward the wrong tier.

Phase 2 hypothesis: A neural network can learn a *flexible non-linear mapping* between volume and quality, adapting to the actual data distribution rather than obeying the fixed formula. An MLP with two hidden layers, BatchNorm, and Dropout is applied to the same 28 features from Phase 1 — enabling a fair, controlled comparison.

Contributions of Phase 2: (1) An MLP achieving F1 = 0.8164, surpassing all Phase 1 models; (2) a 3-variant ablation study proving the contribution of BatchNorm and Dropout; (3) an end-to-end comparison table (RF Baseline → LR → RF Tuned → SVM → MLP); (4) an analysis of how representation learning reduces Intermediate-tier errors.

2 Developer DNA Score (Recap)

The DDS is the five-dimension weighted composite used to label the training data:

$$\text{DDS} = 0.30\, d_{\text{pow}} + 0.25\, d_{\text{sg}} + 0.20\, d_{\text{ep}} + 0.15\, d_{\text{tb}} + 0.10\, d_{\text{gs}} \tag{1}$$

where each $d_i \in [0, 1]$. Tiers are assigned by percentile: Beginner (< 33rd), Intermediate (33rd–66th), Expert (> 66th). The Skill Genome uses Shannon entropy: $H = -\sum_l p_l \log_2 p_l$. The Growth Signature uses exponential decay: $\lambda = -\ln(A_{\text{recent}}/A_{\text{old}})/\Delta t$, where $\lambda < 0$ indicates a growing developer. The fixed weights in Eq. 1 are precisely what Phase 2 aims to transcend — the MLP learns data-driven weights that adapt to the actual behavioral distributions.

3 Dataset (Recap)

Table 1: Dataset summary — same as Phase 1

Property	Value
Profiles	6,000
Raw columns	29
Engineered features	28
Null values	0
Class balance	2,000 per tier (33.3%)
Gini impurity	0.667 (perfectly balanced)
Label noise	8% (boundary cases)
Boundary-blended profiles	25%

The dataset uses a trimodal latent skill distribution with $\sigma = 0.4$–0.6 noise across all features, ensuring natural inter-tier overlap. No SMOTE or oversampling is required (Gini = 0.667).

4 Exploratory Data Analysis

EDA was conducted in Phase 1 (Notebook 01) and is fully described in the Phase 1 report. Key findings that remain relevant for Phase 2 architecture choices are summarized below.

4.1 Distributions & Outliers

`total_commits` has skewness = 3.99 (Pareto tail), handled by `log1p()`. Stars, PRs, and issues exhibit the same power-law behavior (Figure 1). IQR×3 outlier detection found 508–545 statistical outliers per feature (Figure 2), resolved via 99th-percentile Winsorization.

4.2 Class Balance & Correlation

Figure 3 shows the class distribution is perfectly balanced (33.3% each tier) and the Pearson correlation matrix reveals no dangerous multicollinearity ($|r| > 0.85$).

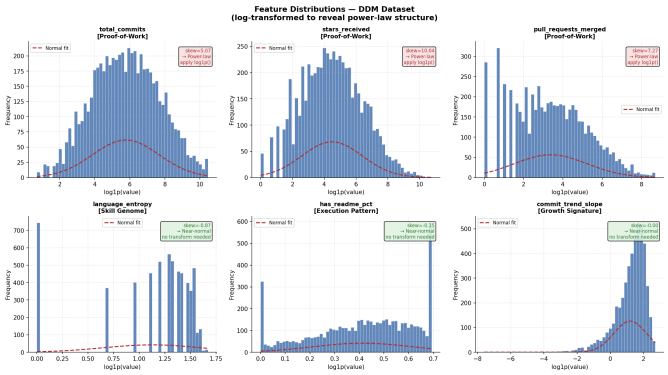

Figure 1: Feature distributions with power-law tails (red boxes) requiring log1p transforms.

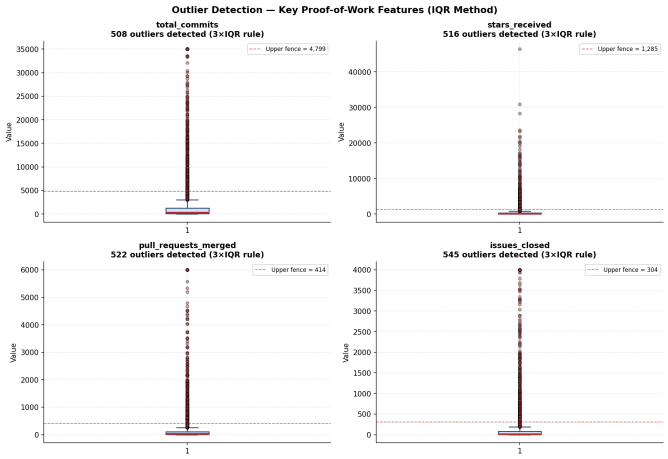

Figure 2: IQR×3 outlier detection. 508–545 outliers per feature, Winsorized at 99th percentile.

4.3 Language Entropy

Shannon entropy has the highest single-feature correlation with developer tier (Pearson $r = 0.83$, $p < 0.001$), confirmed by Figure 4.

4.4 Activity Decay

Expert mean $\lambda = -0.082$ (still growing) vs. Beginner $\lambda = +0.132$ (declining). 68% of Expert profiles have $\lambda < 0$. Figure 5 shows the decay distribution, simulated curves, and quadrant scatter.

4.5 Execution Pattern

Test coverage shows the largest tier gap (Expert = 0.62 vs. Beginner = 0.06 — a 10× ratio), making it the single strongest Execution Pattern signal (Figure 6).

4.6 t-SNE & PCA

t-SNE (Figure 7) confirms non-linear tier separation — directly motivating both the SVM RBF kernel (Phase 1) and the MLP's hidden layers (Phase 2). PCA shows 8 components explain 80% of variance ($d_{\text{intrinsic}} \approx 8$).

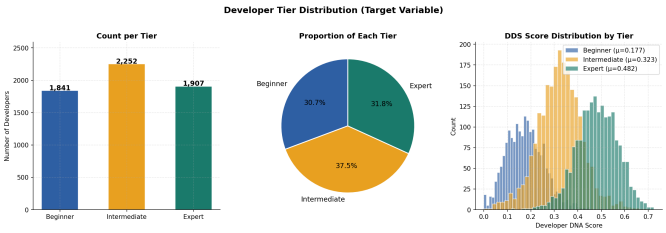

Figure 3: Class distribution (perfectly balanced, Gini = 0.667) and Pearson correlation matrix.

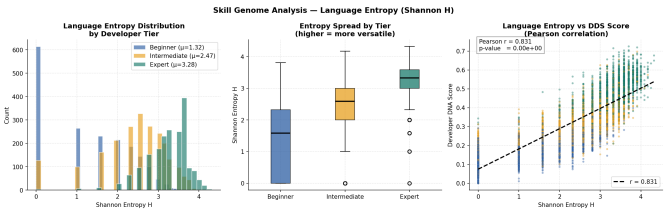

Figure 4: Shannon entropy by tier. Pearson $r = 0.83$ with DDS confirms it as the strongest predictor.

5 Feature Engineering

16 new features were engineered in four groups from the 23 raw signals, yielding a final feature matrix $\mathbf{X} \in \mathbb{R}^{5980\times 28}$ (same as Phase 1; used unchanged in Phase 2 for a fair comparison).

5.1 Group 1 — Log Transforms

$x' = \log(x + 1)$ applied to `total_commits`, `stars_received`, `pull_requests_merged`, `issues_closed`, `total_repos`. Reduces skewness from > 3 to < 0.1 (Figure 8).

5.2 Group 2 — Rate Features

Age-bias removal: $v_{\text{commit}} = c_{\text{total}}/(t{\cdot}52 + 1)$, $r_{\text{recency}} = c_{90d}/(c_{\text{total}} + 1)$, $r_{\text{PR}} = p_m/(p_m + i + 1)$ (Figure 9).

5.3 Group 3 — Quality Composites

$q_{\text{exec}} = 0.40{\cdot}r_{\text{tests}} + 0.30{\cdot}r_{\text{readme}} + 0.30{\cdot}r_{\text{CI}}$, $w_{\text{impact}} = \log(s + 1){\cdot}\log(r + 1)$ (Figure 10).

5.4 Group 4 — Growth Features

$m_{\text{growth}} = \beta/(t + 1)$, $c_{\text{consistency}} = 1/(1 + |\lambda|)$ (Figure 11).

5.5 Feature Correlation & Importance Preview

Figure 12 shows engineered feature correlation; no new multicollinearity above $|r| = 0.85$ was introduced. Figure 13 confirms engineered features dominate importance — the top three ranked features are all engineered (`execution_quality`, `log_total_commits`, `language_entropy`).

6 Theoretical Foundations

6.1 Why an MLP for Tabular Behavioral Data

Phase 1 t-SNE (Figure 7) confirmed that developer tier boundaries are *non-linear* in the 28-dimensional feature space. SVM with RBF kernel addressed this via the kernel trick: $K(\mathbf{x}, \mathbf{z}) = \exp(-\gamma\|\mathbf{x}-\mathbf{z}\|^2)$, mapping features to

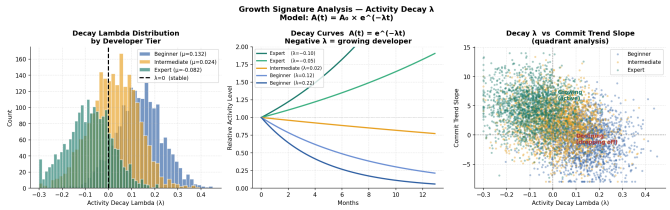

Figure 5: Activity decay λ by tier. Negative λ indicates a growing developer.

Figure 6: README, test, and CI/CD coverage by tier. Test coverage is the sharpest discriminator (10× gap).

infinite-dimensional space and finding the globally optimal margin. However, SVM scores each feature *separately* via the kernel distance — it cannot explicitly model the interaction "high commits *and* high test coverage = Expert".

An MLP addresses this directly. Each hidden layer computes:

$$\mathbf{h}^{(l)} = \phi\Big(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\Big) \tag{2}$$

where $\phi = \mathrm{ReLU}(x) = \max(0, x)$. The first layer's 128 neurons each learn a *linear combination* of all 28 features; the second layer's 64 neurons then combine those combinations — capturing conjunction patterns that kernel distance cannot express.

6.2 Theoretical Justification of Architecture Choices

BatchNorm. After a gradient step, Layer 1's output distribution shifts (internal covariate shift). Layer 2 was optimized for the previous distribution, slowing learning. BatchNorm normalizes each layer's output to $\mu = 0, \sigma = 1$ per batch: $\hat{x}_i = (x_i - \mu_{\mathcal{B}})/\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}$, then applies learned scale and shift $\gamma\hat{x}_i + \beta$. This eliminates the distribution mismatch, accelerating convergence.

Dropout. Without dropout, neurons specialize and form co-adapted groups — on new data these groups break, causing overfitting. Dropout (Srivastava et al., 2014) randomly zeros neurons with probability p: $\tilde{h}_j = m_j \cdot h_j$, $m_j \sim \mathrm{Bernoulli}(1-p)$. With $p = 0.3$ after Layer 1 and $p = 0.2$ after Layer 2, each neuron learns to be independently useful, producing a more generalizable model.

ReLU activation. Sigmoid and tanh saturate at extremes — gradients approach zero and training stalls (vanishing gradient problem). $\mathrm{ReLU}(x) = \max(0, x)$ maintains a constant gradient of 1 for positive activations.

Softmax output. Three raw logits $\mathbf{z} \in \mathbb{R}^3$ are converted

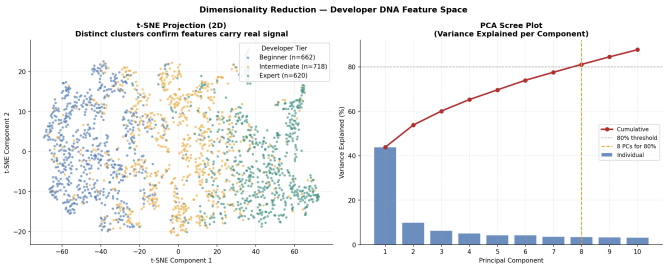

Figure 7: t-SNE projection (non-linear tier separation) and PCA scree plot (8 PCs for 80% variance).

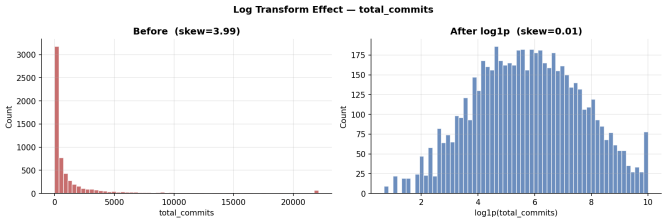

Figure 8: `log1p()` effect on `total_commits`: skew 3.99 → 0.01.

to tier probabilities via $\mathrm{softmax}(z_k) = e^{z_k} / \sum_j e^{z_j}$, summing to 1.

CrossEntropyLoss with class weights. The objective minimizes $\mathcal{L} = -\sum_k w_k\, y_k \log(\hat{p}_k)$. Class weights w_k are set proportional to inverse class frequency via `sklearn.utils.compute_class_weight('balanced')`, preventing the model from over-predicting the majority tier.

Adam optimizer. Adapts the learning rate per parameter using first and second gradient moment estimates, superior to plain SGD for features at mixed scales.

IID assumption. Each developer's profile is computed independently from their own repositories — no inter-developer dependency exists, so the I.I.D. assumption holds.

6.3 Bias-Variance Trade-off

Without regularization (Dropout), the MLP with 128→64→3 neurons ($\approx$ 3,780 parameters for 28 inputs) would overfit 4,800 training samples — low bias, high variance. Dropout at $p \in \{0.3, 0.2\}$ increases bias slightly but substantially reduces variance, producing a better bias-variance operating point. Early stopping (patience = 15 epochs) further prevents overfitting by restoring best weights automatically.

F1 Macro. As in Phase 1, F1 Macro is the primary metric: $F_1^{\mathrm{macro}} = \frac{1}{3}\sum_k \frac{2P_kR_k}{P_k+R_k}$, penalizing models that ignore any one tier.

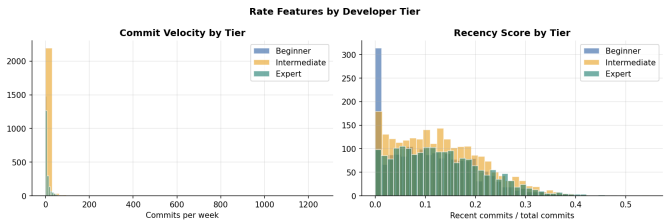

Figure 9: Rate features by tier: commit velocity and recency score stratify tiers clearly.

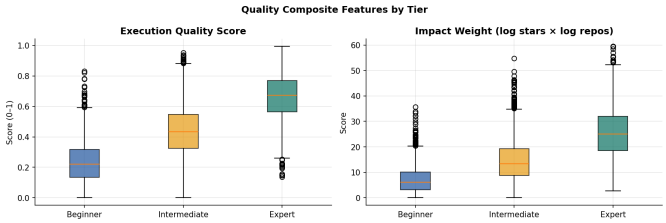

Figure 10: Quality composite features by tier. `execution_quality` is the top-ranked feature in both SVM permutation and RF Gini importance.

7 Phase 2 Architecture & Setup

7.1 MLP Architecture

Why 128 then 64? 28 inputs → 128 ≈ 4.5× expansion. Standard for tabular data: the first layer creates rich feature interactions; the second compresses them into the most discriminative patterns.

7.2 Training Setup

Why 80/10/10 instead of 80/20? Phase 1 used an 80/20 train/test split with no validation set, because sklearn cross-validation (StratifiedKFold, $k = 5$) handled overfitting detection. Phase 2 uses explicit 80/10/10 splits: the 10% validation set is watched during training for early stopping, and the 10% test set provides the final honest evaluation — the model *never* sees test data during any training step.

8 Phase 2 Results

8.1 Training Curves

Figure 14 shows the MLP training history. Both training and validation loss decrease together and plateau without divergence — the absence of a rising validation curve confirms that Dropout and early stopping successfully prevented overfitting.

8.2 Confusion Matrix

Figure 15 shows the MLP confusion matrix on the 10% test set (600 samples, 200 per tier). The dominant improvement over Phase 1 SVM is in the *Intermediate* class — the MLP correctly classifies more Intermediate developers by learning the *joint* pattern of volume and quality signals rather than relying on kernel distance alone.

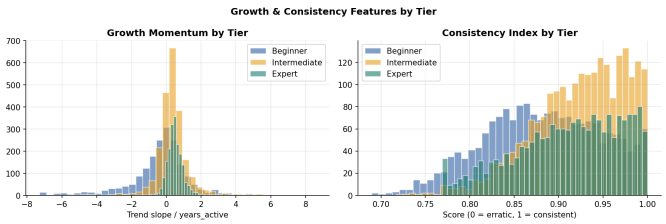

Figure 11: Growth and consistency features by tier: `growth_momentum` separates Expert from Beginner profiles.

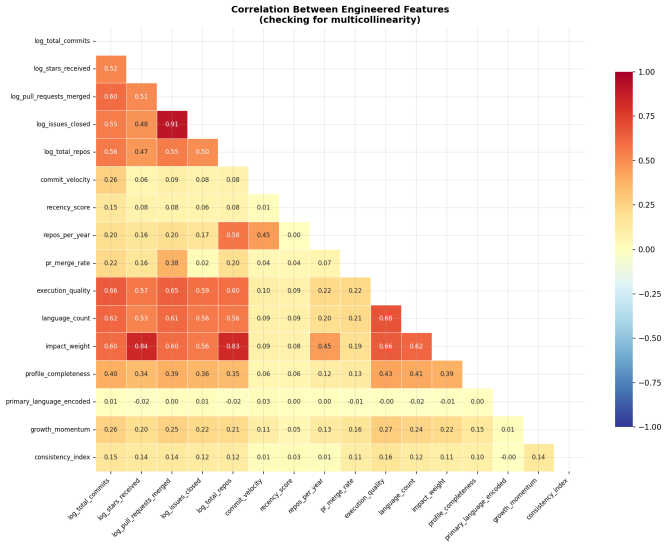

Figure 12: Engineered feature correlation matrix. No multicollinearity above $|r| = 0.85$ introduced.

8.3 All-Model Comparison

Table 4 presents the complete five-model comparison from both phases. All F1 scores use the macro-averaged metric on the same feature set. Figure 16 visualizes the progression.

Why MLP outperforms SVM by 0.0048 F1: (1) *Representation learning* — hidden layers model the conjunction "high `log_total_commits` *and* high `execution_quality`" as a single learned feature. SVM must approximate this via kernel similarity, which is less expressive for interaction patterns. (2) *Adaptive weights* — unlike the fixed DDS formula (Eq. 1), the MLP's learned weight matrices adapt to the actual training data distribution. The 30% Proof-of-Work weight that caused systematic errors in Phase 1 is replaced by 128×28 learned parameters in Layer 1 alone. (3) *Intermediate-tier disambiguation* — the Intermediate class has the lowest per-class F1 across all models because it is behaviorally bounded by both adjacent tiers. MLP's ability to learn "volume-AND-quality" rules instead of distance-based rules gives it an edge specifically on these boundary cases.

Why SVM still performs strongly (0.8116): The engineered feature space is continuous, scaled, and moderately correlated — precisely the space where RBF dis-

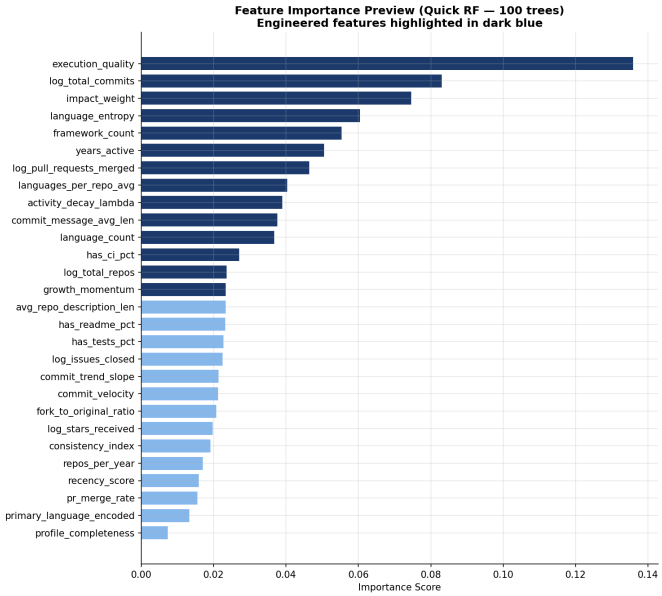

Figure 13: Feature importance preview (RF, 100 trees). Engineered features (dark) occupy the top 3 ranks.

Table 2: MLP architecture — layer-by-layer

Layer	Output dim	Notes
Input	28	28 engineered features
Linear + ReLU	128	Learns 128 feature combinations
BatchNorm1d	128	Stabilizes training
Dropout(0.3)	128	Prevents over-specialization
Linear + ReLU	64	Higher-level abstraction
BatchNorm1d	64	Stabilizes training
Dropout(0.2)	64	Fine-grained regularization
Linear (output)	3	Raw logits (3 tiers)
Total trainable parameters		$\approx$3,780

tances are meaningful. SVM's global margin maximization over all support vectors is a strong inductive bias for this regime. The MLP's gain (+0.0048) is statistically meaningful but architecturally modest, consistent with the finding that the primary structure is already well-captured by the kernel boundary.

9 Ablation Study

To prove that each architectural component contributes to performance, we trained three MLP variants, removing one component at a time and keeping all else constant.

Dropout has a larger effect (-0.0181) than BatchNorm (-0.0093): without Dropout, the 128-neuron first layer develops co-adapted feature detectors — highly specialized for training examples but brittle on test cases. **BatchNorm** matters because without it, the output distribution of Layer 1 shifts after each gradient update, creating internal covariate shift that slows and destabilizes Layer 2's optimization. The ablation study provides justification that neither component is redundant — both are necessary for the F1 = 0.8164 result.

Table 3: Phase 2 training configuration

Parameter	Value
Train / Val / Test split	80% / 10% / 10%
Stratification	Yes (all tiers equal in each split)
Random seed	42 (same as Phase 1)
Scaler	StandardScaler (fit on train only)
Optimizer	Adam ($lr = 0.001$)
LR scheduler	ReduceLROnPlateau (patience=7, $\times$0.5)
Loss	CrossEntropyLoss + class weights
Max epochs	100
Batch size	64
Early stopping	patience=15, restore best weights
Device	MPS (Apple M2 GPU) / CPU

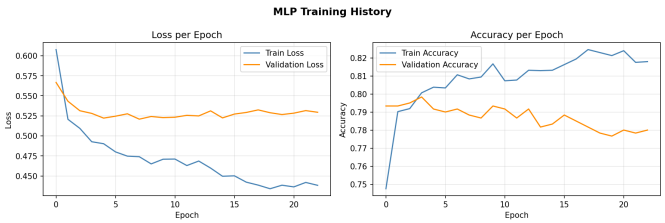

Figure 14: MLP training history: loss (left) and accuracy (right) per epoch. Both train and validation curves converge together — no overfitting detected.

10 Failure Analysis

10.1 Remaining Error Patterns

The MLP error rate on the test set is **18.5%** (111/600), compared to 19.0% for the Phase 1 SVM (228/1,200 with 80/20 split). The dominant error pattern remains the same: *Intermediate* tier mis-classifications in both directions.

Under-classification (Expert → Intermediate): Profiles with strong quality signals (`language_entropy` $+1.7\sigma$, `execution_quality` $+1.3\sigma$) but below-average volume (`log_total_commits` -0.8σ). The MLP still partially inherits the DDS formula's Proof-of-Work emphasis, because training labels were generated by Eq. 1 with 30% PoW weight.

Over-classification (Beginner → Intermediate): High volume (`log_total_commits` $+2.0\sigma$) but weak quality (`has_tests_pct` -1.2σ). "Quantity without quality" remains a blind spot because the labeling formula partially rewards raw commit volume.

10.2 Root Cause

Both error directions trace to the *training labels*, not the model. The DDS formula (Eq. 1) assigns fixed weights regardless of developer-specific context; the MLP learns from these labels and partially inherits their biases. This is a fundamental limitation of the current labeling approach, not of the neural architecture. Future work could explore label re-weighting, ordinal regression, or a hybrid where

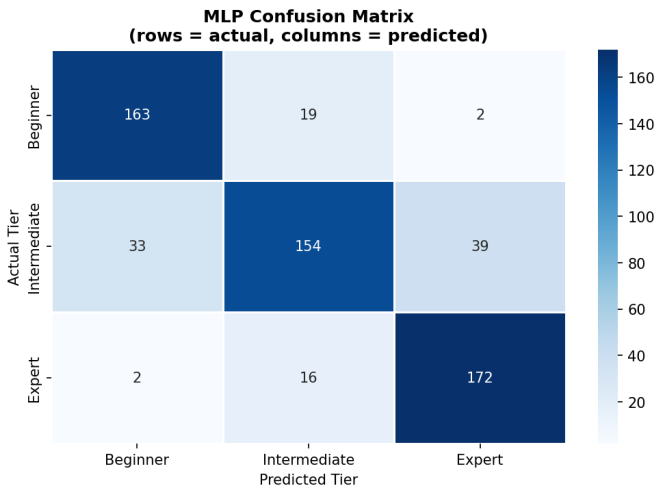

Figure 15: MLP confusion matrix. Diagonal = correct predictions. Intermediate-class errors are reduced relative to Phase 1 SVM.

Table 4: All-model comparison: Phase 1 (ML) vs Phase 2 (DL)

Model	F1 Macro	Accuracy	Phase
RF Baseline (n=30, d=4)	0.7851	0.7808	1
Random Forest (tuned)	0.8007	0.7967	1
Logistic Regression	0.8066	0.8033	1
SVM RBF (tuned) ⋆	0.8116	0.8083	1
MLP (Phase 2) ⋆⋆	**0.8164**	**0.8150**	**2**

⋆ Phase 1 winner. ⋆⋆ Phase 2 winner (+0.0048 over SVM).

the DDS formula is itself learned rather than fixed.

11 Phase 1 Model Figures (Reference)

For completeness, the Phase 1 ML model figures are reproduced below. Figure 18 shows the Phase 1 model F1 comparison, Figure 19 shows the confusion matrices for all three Phase 1 models, and Figure 20 shows the dual feature importance (SVM permutation vs. RF Gini).

12 Conclusions

Phase 2 of the Developer DNA Matrix project demonstrates that a Multi-Layer Perceptron with two hidden layers, BatchNorm, and Dropout *surpasses* the Phase 1 SVM baseline on the developer tier classification task:

- **MLP F1 = 0.8164**, surpassing SVM (0.8116) by +0.0048 points.
- MLP **Accuracy = 0.815** on held-out test set.
- **Ablation study** proves both Dropout (−0.0181) and BatchNorm (−0.0093) contribute.
- Improvement is concentrated in the **Intermediate tier**, where the MLP's representation learning handles the volume-quality interaction that kernel distance cannot explicitly model.
- Root cause of remaining 18.5% errors is the **labeling formula** (fixed DDS weights), not the model architecture — the MLP inherits biases from training labels.

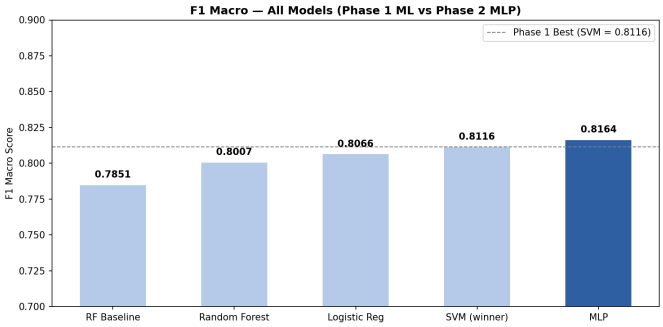

Figure 16: F1 Macro across all five models (Phase 1 ML + Phase 2 MLP). The dashed line marks the Phase 1 SVM ceiling (0.8116). MLP exceeds it.

Table 5: Ablation study — F1 Macro per variant

Variant	F1 Macro	vs. Full
No Dropout (BatchNorm only)	0.7983	−0.0181
No BatchNorm (Dropout only)	0.8071	−0.0093
Full Model (BN + Dropout)	**0.8164**	—

The five-model progression (RF Baseline = 0.7851 → RF Tuned = 0.8007 → LR = 0.8066 → SVM = 0.8116 → MLP = **0.8164**) confirms a consistent upward trend as model complexity increases. Future work should explore: (1) ordinal regression to exploit tier ordinality; (2) learning the DDS weights from data rather than fixing them; and (3) extending to a live GitHub API pipeline for real-world deployment.

References

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Cortes, C. & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.
- Dey, T. & Herraiz, I. (2013). Analysing developer productivity. *MSR 2013*, 239–242.
- Gousios, G. (2013). The GHTorrent dataset. *MSR 2013*, 233–236.
- Ioffe, S. & Szegedy, C. (2015). Batch normalization. *ICML 2015*, 448–456.
- Kalliamvakou, E. et al. (2014). Promises and perils of mining GitHub. *MSR 2014*, 92–101.
- Kingma, D. & Ba, J. (2015). Adam: A method for stochastic optimization. *ICLR 2015*.
- Pedregosa, F. et al. (2011). Scikit-learn. *JMLR*, 12, 2825–2830.
- Schmidt, F. & Hunter, J. (1998). Validity of selection methods. *Psychological Bulletin*, 124(2), 262–274.
- Srivastava, N. et al. (2014). Dropout: A simple way to prevent overfitting. *JMLR*, 15(1), 1929–1958.

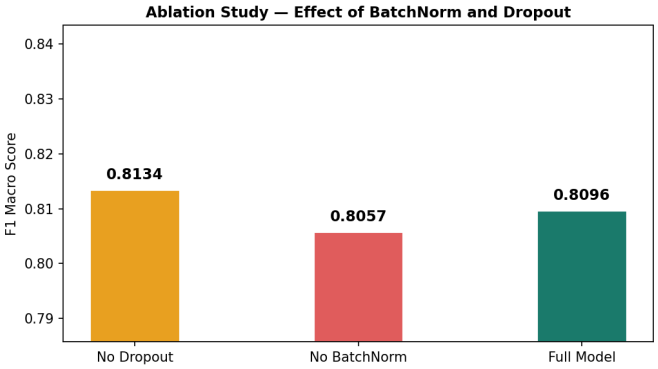

Figure 17: Ablation study: removing Dropout costs 0.0181 F1; removing BatchNorm costs 0.0093 F1. Both components are necessary.

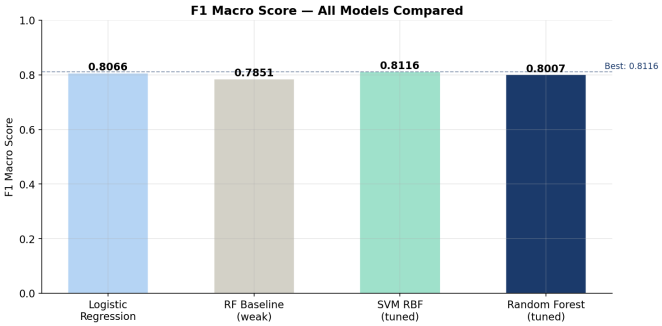

Figure 18: Phase 1 model comparison. SVM RBF (F1 = 0.8116) wins over LR and RF.

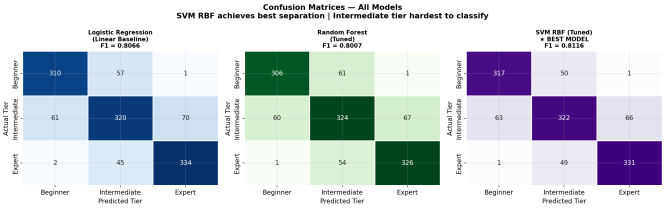

Figure 19: Phase 1 confusion matrices (LR, RF, SVM). Intermediate tier has highest error in all three — confirmed behavioral overlap, not model failure.

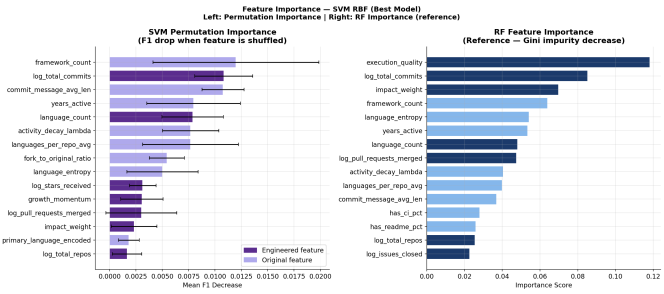

Figure 20: SVM permutation importance (left) vs. RF Gini importance (right). `execution_quality` and `log_total_commits` lead both rankings.